CHALLENGE SHEET

Parcelpoint

Investigate a fictional courier API and recover two flags through its parcel-release workflow.

DIFFICULTY	TRACK	HOST PORT	LAB ID
MEDIUM	WEB	8080	parcelpoint

EASY · **MEDIUM** · HARD · INSANE

VERIFY THE SOLVE

```
python3 ../scripts/check.py parcelpoint
```

Brief

ParcelPoint is a **medium Web/API** lab. A browser console and a documented JSON API sit in front of the locker network. Your goal is to recover the checkpoint flag, then the final release flag for the held parcel.

The scenario, people, addresses, parcels, and records are fictional. Use only the supplied local lab instance.

Setup

Prerequisites:

- Docker Desktop using Linux containers, or Docker Engine on Linux, with Docker Compose `v2.17.0` or later.
- Python 3.10 or later on the host for hash synchronization, reset, and the local checker; no additional Python packages are required.
- A browser or an HTTP client, and an available local port `8080`.
- The first build requires internet access to download the pinned base image and Python packages. The running lab needs no external service.

Run the commands from this lab directory. Substitute `python3` for `python` if needed, for example on Linux and macOS.

```
docker compose up -d --build --wait
python scripts/sync_flag_hash.py
```

Open the UI at `http://127.0.0.1:8080`.

Read the player guide for the lab workflow.

The only published host port is `127.0.0.1:8080`. The application and its SQLite database are not published directly.

To reset the instance and refresh both checker digests:

```
python reset.py
```

The reset rotates accounts, sessions, reset codes, and both flags. Any flag captured before the reset is no longer valid. The reset does not pull images or rebuild.

To stop the lab and remove its containers and temporary state:

```
docker compose down
```

Starting again creates a fresh instance; run `python scripts/sync_flag_hash.py` again after a fresh start.

Goal

Find the checkpoint flag first, then the final release flag. Both use the form `duck{...}`. Check each captured flag from this directory:

```
python ../../../scripts/check.py .
```

The checker prompts for a flag, prints `checkpoint solved` or `solved`, and does not print the submitted value. It stores and compares only the SHA-256 digests in `lab.yml`.

Notes

- This lab is intentionally vulnerable. Do not apply the technique to systems you do not own or have permission to test.
- Treat the lab as a black box: only the web UI and API at `127.0.0.1:8080` are in scope. The lab directory also holds the source code and maintainer documentation, including the official solution; reading them spoils the challenge.
- The host, Docker, database, images, and reset controls belong to the operator and are not challenge assets. In hosted delivery, only the assigned HTTP(S) endpoint is in scope.